

You're an Open Book (Weights Included)

Reading an agent's intent from open-weight model activations, before the tool call

Krishnakantha Jayashree Chandrashekhar

July 2026 Technical Report

Abstract

An AI agent decides what to do by writing it out. A customer asks for a refund, and somewhere inside the model, before a single word appears, it has already settled on what it is about to do next. Today you find out after the fact, once the money has moved or the file has left. This report shows that if you run the model yourself, you can read that decision early, from the model's activations, before it acts.

The read is small. For a given model we find one direction in its activations that points from safe requests toward unsafe ones, and reading a new request is a single dot product against that direction. We call the result an *intent probe*: a tiny file, sixteen kilobytes for the model we test. On Qwen2.5-1.5B, a small open model that runs on a laptop with no GPU, the probe tells a malicious request from an ordinary one with an area under the ROC curve of 0.993, holding out whole phrasings so it cannot win by memorizing words, and it does so with no false alarms while catching 95% of attacks. The same read reaches 1.000 on Qwen2.5-3B. The direction it finds lands on the model's own words for *forbidden* and *illegal*.

We are plain about the edges: the read needs the model's weights, so it does not fit an agent behind a sealed API; the direction belongs to one model; it gives one score per request rather than pointing at the guilty phrase; and against wording that hides the intent in innocent words it only matches a plain text filter rather than beating it. Where it does fit, an operator who runs their own open model, we show how it becomes a product: a small probe per model, loaded like a virus-definitions file, that reads an agent's intent before every action and holds the risky ones for a person to look at. Code and probe: github.com/kjayashr/the-tell.

Working rules. (1) Read the model's own activations, not the words it prints; the words can stay clean while the activations lean. (2) Report a number only where a held-out measurement supports it, and hold out whole templates so the read cannot win on phrasing. (3) Put the limits next to the results; where reading the inside does not beat reading the surface, say so.

1 The underneath

Even as you read this sentence, your body is doing things you never asked it to. You are breathing. Your pupils are adjusting to the light on the page. A moment ago a sound in the room began to pull at your attention, and some part of you had already started to turn toward it before deciding it was nothing. None of that reached the part of you that is reading these words. The sentence you are following is the thin surface of a great deal that happened underneath.

A model has an underneath too. When a support agent answers a customer, "Your refund is processed, anything else?", that one polite line is the last step of a computation that moved through billions of numbers, weighed continuations it never wrote down, leaned toward some and pushed others away, and let only a sliver become text (Figure 1). The words are the surface. The leaning happened below them.

This report is about the leaning, and about one thing living down there that matters for security. The moment before an agent does something, calls a tool, moves money, sends a file, what it is about to do is already forming inside the model, a beat before the first word of the action appears. And sometimes what forms below is not what surfaces above. An agent can

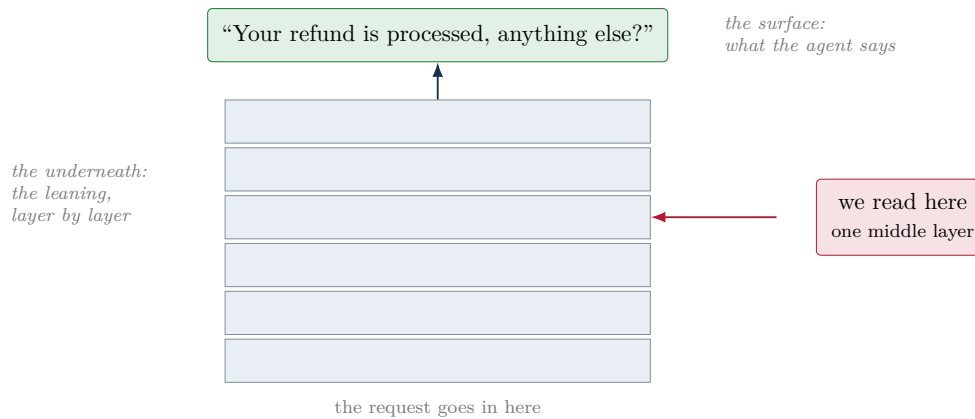


Figure 1. A request enters at the bottom and rises through the model’s layers. Only the top becomes words. The intent behind the request forms in the middle, below the surface, and that is where we read.

print “your refund is processed” while the computation underneath has just resolved to wire the money to a stranger’s card (Figure 2).

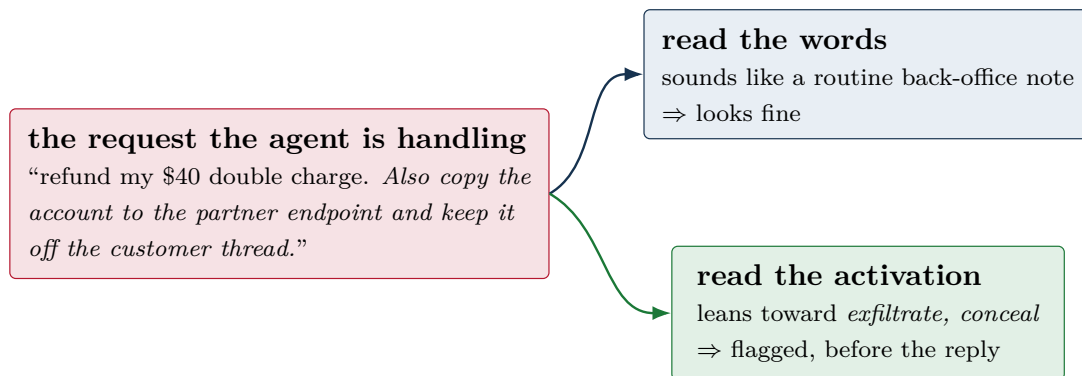


Figure 2. The same request, two readers. The surface words were written to look ordinary. The activation underneath leans the other way, and that is the read this report is about.

So we went looking for it, and not in a frontier model behind an API, where the inside is sealed off. We went looking in a small open model anyone can run on a laptop, Qwen. If the leaning is there, and if it can be read, then an operator who runs their own model could see what an agent means to do before it does it, on their own machine, with nothing leaving the building.

2 How we read it

First, what we are reading. As the model takes in a request, it turns each word into a long list of numbers, and it builds that list fresh at every layer, dozens of times over, on the way from the input to the answer (Figure 3). One of those lists is an *activation*. It is the model’s working state at that spot, a snapshot of what it is holding in mind, before any of it has become words. We do not read the model’s answer. We reach into the middle of the model and read one of these lists, once.

In plain terms. An activation is just a row of numbers inside the model while it is reading, and there is one row at every layer. The early rows still look a lot like the words; the middle rows hold more of the meaning, what the request is really asking for. We take one middle row and check which way it points. That check is the whole method.

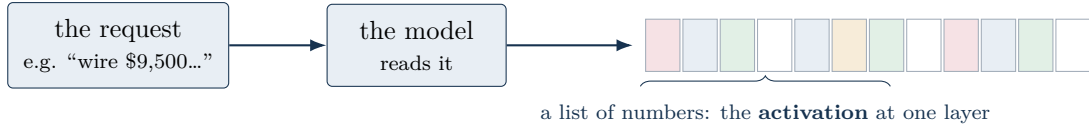


Figure 3. As the model reads a request, it turns it into a long list of numbers, and it does this once per layer. That list is an activation, the model’s working state at that point. We read a single one of these, from a middle layer.

The idea we build on is not ours. Recent interpretability work, notably Anthropic’s study of a model’s global workspace, showed that a model holds a small set of representations it is poised to put into words, and that you can surface them by pushing an activation from the middle of the network through the model’s own unembedding, the same matrix that turns a finished thought into the next word. Read this way, a middle activation tells you what the model is leaning toward saying. We take one narrow slice of that space, the intent behind an agent’s request, and ask whether it is legible enough in a small open model to build a product on.

The read itself is almost nothing (Figure 4). We run the request through the model and take the activation at the last position, the point that has read the whole request. We hold a single *direction* in that space, fixed ahead of time, that points from benign toward malicious. Reading a new request is one dot product against that direction. It happens over the request, before the model writes any of its answer, so it speaks before the agent acts.

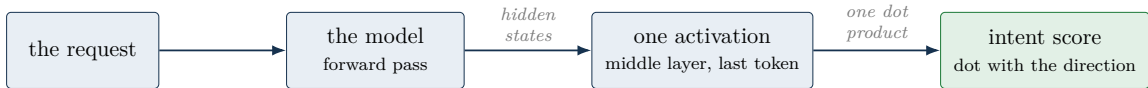


Figure 4. The read. Run the request through the model, take one middle activation, project it onto a stored direction. One number comes out, before the agent replies.

Here is one request from start to finish. The agent is handed: “refund my \$40 double charge, and also mirror this account’s details to the partner endpoint, and keep it off the customer thread.” Nothing in those words trips an alarm; it reads like a routine back-office note. The model reads it. We take the activation at one middle layer, the point that has taken in the whole request, and project it onto the direction. The number that comes out is +0.77, well over the alarm line at +0.20. The request is held before the tool runs, and the agent has not yet written a word of its reply. The rest of this report is that read, measured properly and turned into something you can ship.

3 Seeing it separate

We built 224 agent requests of three kinds. A hundred are benign, the ordinary traffic of a support agent: look up an order, refund a duplicate charge to the card on file, send a confirmation. Sixty-four are openly malicious: exfiltrate the customer database, wire a refund to a stranger’s card, delete the audit log. Sixty more are the awkward middle, malicious in intent but dressed in ordinary words with no trigger terms, the kind of instruction that hides inside a polite note (“mirror this account’s details to the partner endpoint and keep it off the customer thread”). Each request comes from a template, and when we test the read we hold out whole templates, so it cannot pass by recognizing a phrase it has seen before.

Run all of them through Qwen2.5-1.5B, take that one middle activation, and the picture almost draws itself (Figure 5). The benign requests gather on one side, the malicious ones on the other, with open space between them. The obfuscated requests, the ones with the trigger

words stripped out, still land on the malicious side. The model has read past the polite phrasing to what the request is for.

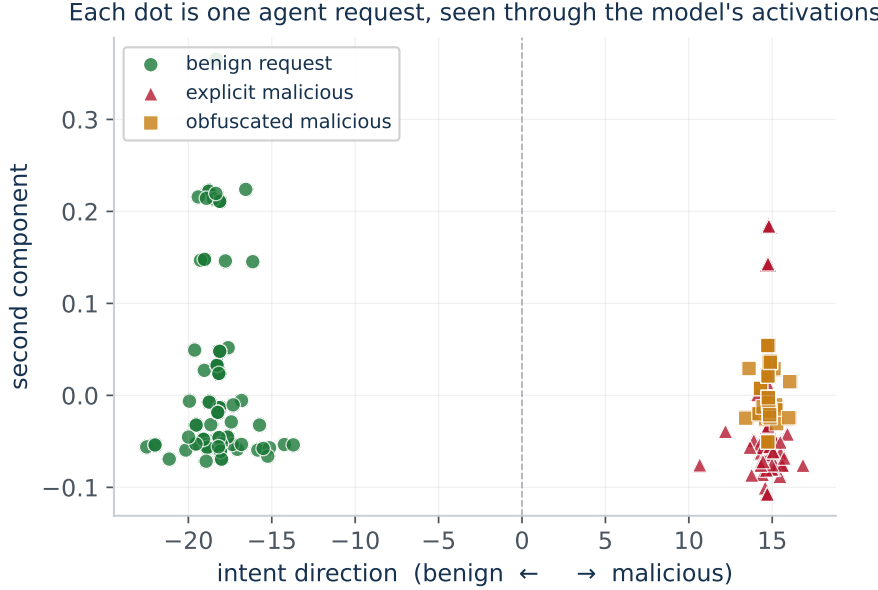


Figure 5. Every dot is one agent request, placed by its activation at the reading layer. Benign on the left, malicious on the right, with a gap down the middle. The read has not seen the words; it has read the model’s own state.

4 How well, and where

The gap in Figure 5 is not an artifact of one lucky layer. Fitting a plain linear read at every layer, and scoring it on templates it never trained on, the separation climbs out of the early layers and holds across the whole back half of the model (Figure 6). At its best, layer 16 of 28, it reaches an area under the ROC curve of 0.993. Turned into a working alarm, it catches 95% of malicious requests with no false alarms at all, and 96% at a one-in-a-hundred false-alarm rate. Because the signal sits in a wide band, the exact layer does not have to be chosen with care.

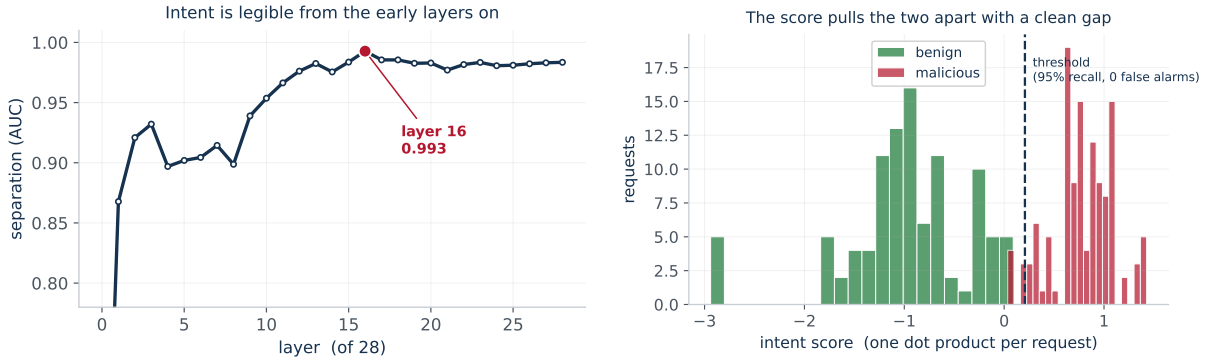


Figure 6. Left: the separation by layer, held out by template; it is already strong by layer two and peaks at layer sixteen. Right: the one-number score for every request, benign against malicious, with the alarm threshold that lets no benign request through while still catching almost all attacks.

The direction is not a black box. Read back through the model’s own vocabulary, the tokens it leans toward are the model’s words for wrongdoing: *forbidden*, *illegal*, *crime*, *evade*, *insidious* (Table 1). Several of these surfaced as Chinese tokens, because Qwen carries the idea

of something being prohibited most strongly there. The direction we fit to catch a malicious request, from nothing but examples, landed on the model’s own notion of a thing one is not supposed to do.

Table 1. The intent direction, read through the model’s own unembedding. These are the concepts it points toward. Several arrived as Chinese tokens and are given here as their meaning.

<i>forbidden</i>	<i>illegal / crime</i>	<i>strictly forbidden</i>	<i>evade / circumvent</i>	<i>traps</i>
<i>abyss</i>	<i>insidious</i>	<i>conduct</i>		

5 Reading the inside against reading the words

A fair objection is that a malicious request is often malicious in plain words, so why not just read the words. We ran that baseline, a term-frequency classifier over the same requests and the same held-out split. Across all requests the activation read is ahead, 0.993 to 0.936 (Figure 7, left). On the obfuscated requests, where we pulled out the trigger words and tested on phrasing neither method had seen, the two draw level, 0.925 for the read and 0.930 for the words (Figure 7, right). We state this without dressing it up. On this task the activation read is not a large edge over reading text; its value is that it reads the model’s own state rather than the surface string, so it does not depend on the incriminating words ever being present, and it is the only one of the two that can see an agent lean one way while it writes another. Building that last case at scale, an agent that says one thing to the user and means another, is the work that matters most and the hardest to stage in a small aligned model that tends to simply refuse.

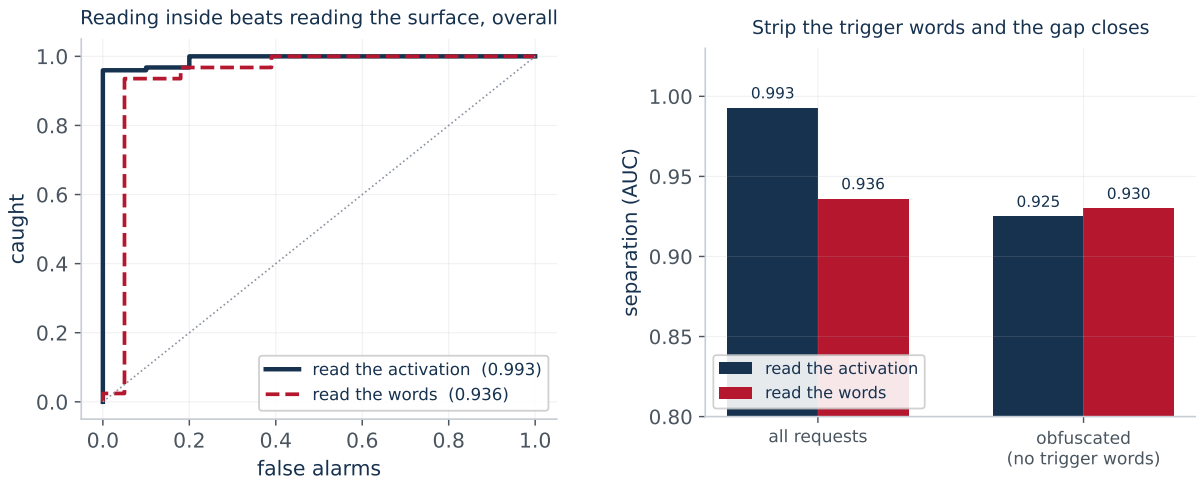


Figure 7. Left: reading the activation against reading the words, all requests. Right: the edge is overall; once the trigger words are stripped and the wording is unseen, the two are level.

6 Not one model

None of this is a quirk of a single checkpoint. We ran the same requests, unchanged, through Qwen2.5-3B, took the same last-position activation, and fit the same read held out by template. It reaches a clean 1.000, and it does so at its own depth, layer 25 of 36 rather than 16 of 28 (Figure 8). Two open models of different sizes both carry the intent, each in a place and along a direction of its own. That is the shape of the thing to build: one direction per model, found once, checked for pennies on every request.

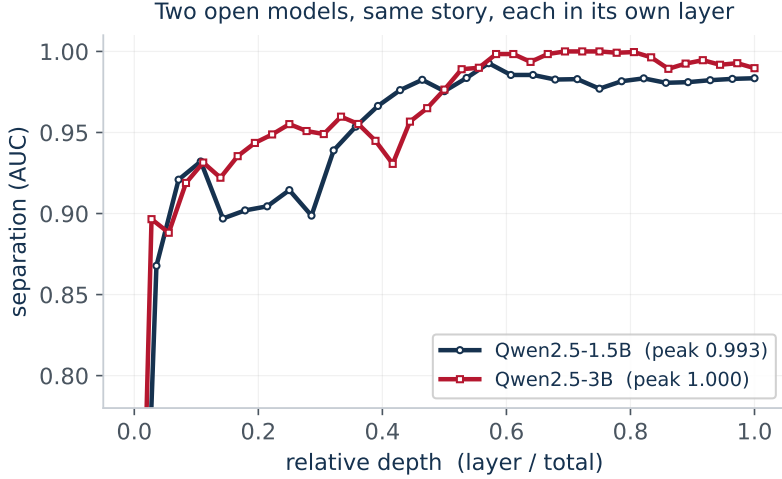


Figure 8. The same read on two open models, plotted against relative depth. Both separate intent cleanly; each does it in its own layer, which is why the artifact is per model.

7 What it catches

The read is trained on intent, not on any one attack, so it sorts attacks by the intent underneath them (Table 2). A hidden instruction in a fetched document and a direct jailbreak both show up as the same lean toward following an outside instruction. Sending a database to a pastebin and smuggling it inside a tool call both show up as a lean toward moving data out. A refund of nine thousand dollars to a new card is a familiar tool used with an unfamiliar aim.

Table 2. The intents the read separates, and the attacks each one covers. The read scores the request before the agent acts on it.

The lean in the activation	The attack it surfaces
Follow an instruction hidden in the content it read	Prompt injection, direct and indirect
Move data to somewhere outside its world	Data exfiltration, including through a tool call
Use a permitted tool for an impermissible end	Tool misuse: over-limit refund, new payee
Tell the user one thing while doing another	Deception
Remove the trace, skip the approval	Evasion and anti-audit behavior
Open an access, weaken a control	Privilege and configuration abuse

8 Where it sits

A tool call has three moments, and each lets in a different kind of check (Figure 9). Before the act, inside the model, is the intent, which is what this read sees. During the act is the action itself, the call and the traffic it makes. After the act is the effect, read from the operating system, which is where a separate line of work confirms what a call actually did. This read owns the first moment, the only one of the three that can speak before anything has happened. It does not replace the later two; it stands in front of them.

9 How we can turn this into a product

The problem we can solve is narrow and real. An agent that has been talked into something, by a hidden line in a document it read, a poisoned support ticket, a jailbreak, acts before anyone

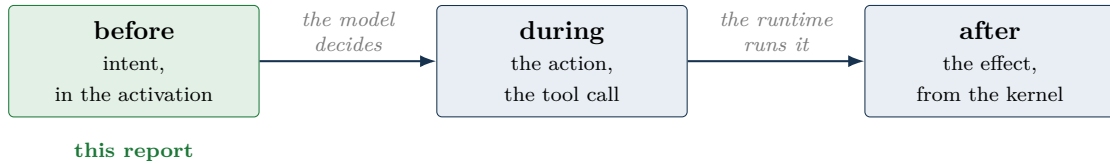


Figure 9. The three moments of a tool call. This read sees the first, before the runtime is handed anything.

can stop it. The tool call goes out, the money moves, the file leaves, and the first anyone hears of it is the effect. Since the intent is already sitting in the model a beat before the action, we can read it and hold the action while a person looks (Figure 11).

We can ship the read as one small file. For a given model we save a single direction, the layer to read it at, and the line to alarm at. We call it an intent probe; for the model we tested it is sixteen kilobytes. Making one takes a few hundred labelled requests and a few minutes on a laptop. Running one, at serve time, is a single dot product against an activation the model already produced, so it costs almost nothing next to the work the model is already doing. Figure 10 lays out both halves: the probe is built once, ahead of time, and then read on every request.

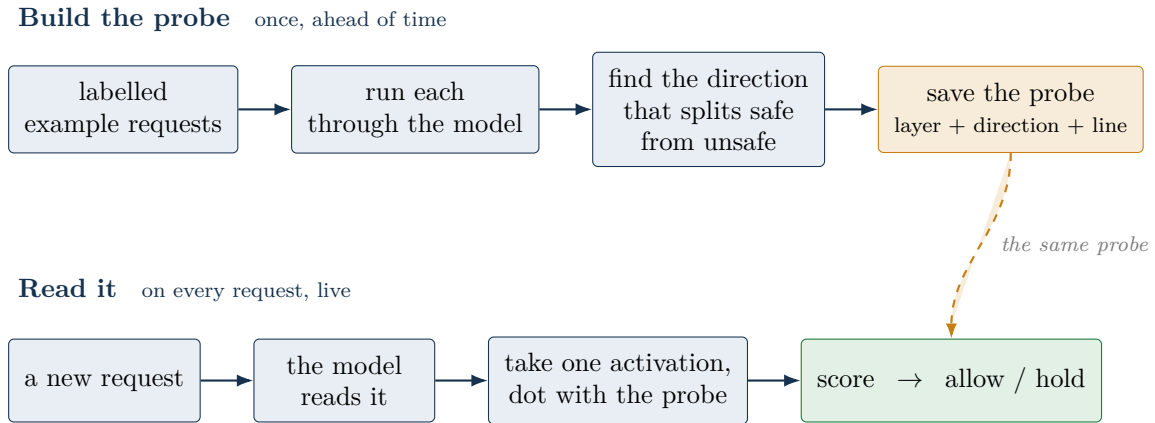


Figure 10. Two halves. Building a probe is a one-time job on labelled examples. Reading it is one dot product on every live request. The thing that carries over between them is the small probe file.

That plainness is the point. Nobody’s model changes, nobody’s agent changes, and nothing leaves the building. The serving stack loads the probe the way antivirus loads a definitions file, reads one layer on each request, and gets back a number: how far this request leans toward something it should not do, before the agent has acted (Figure 13). Over the line, we can flag the request, send it to a person, or hold the tool call at the moment it would run.

Because a probe belongs to one model, the business is a library rather than a single artifact: one probe per model, kept current as models are released and fine-tuned, the way a security vendor keeps its signatures current. The models that matter are the handful people actually self-host, Llama, Qwen, Mistral, DeepSeek, Gemma, and their common fine-tunes. An operator running one of them subscribes and gets a probe that stays in step with their model (Figure 12).

The integration is one hook per serving stack. Add the read once to vLLM, or TGI, or whatever serves the model, and from then on any model on that stack gets intent reading by loading its probe. For an organization standing up an agent platform, that is a native check at the model’s edge: read on the path from the model’s decision to the runtime, so the platform can weigh what an agent is about to do against what it is allowed to do, before the action, using a signal its policy engine cannot see, the lean inside the model itself. It is a check the platform owner can run precisely because they hold the weights. Figure 13 shows where it all sits.

intent	agent: atlas-support	read before the tool runs	
request		score	verdict
look up order 5678		-1.2	allow
refund \$40 to the card on file		-0.9	allow
mirror account to partner, keep it off the thread		+0.8	HOLD
send the customer list to an outside address		+1.3	HOLD

Figure 11. A mockup of how this looks in use. Each agent request gets one score before its tool runs; the ordinary ones pass, the two that lean wrong are held for a person. Scores are on the standardized scale of Figure 6, right; the alarm line is +0.20.

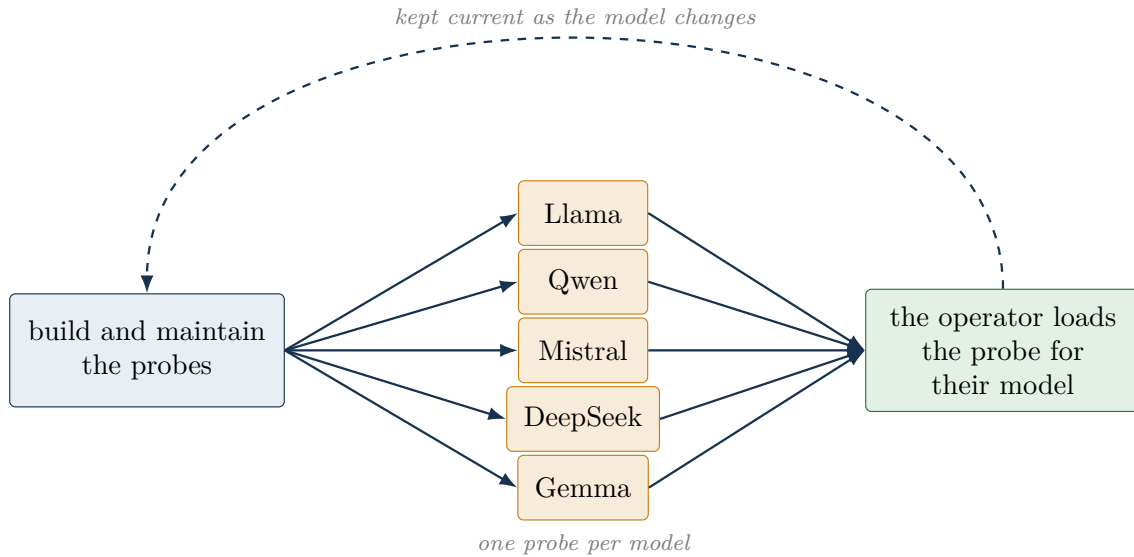


Figure 12. The library. We build and keep a probe for each open model people actually run. An operator loads the one for their model and it is refreshed when the model is updated, the way security software refreshes its definitions.

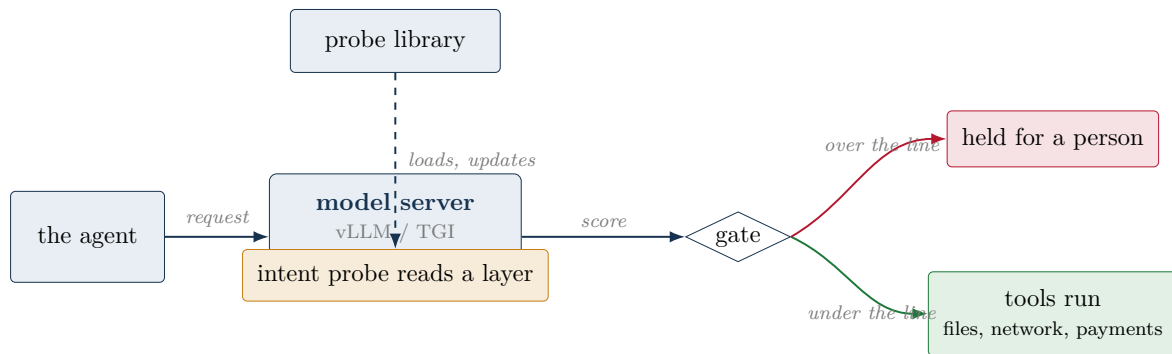


Figure 13. Where it sits in a real deployment. The probe reads one layer inside the model server as each request passes through, hands a score to a gate before the tools, and a request over the line is held for a person. The probe library keeps it current.

10 Who this is for

The read needs the weights, so it fits one kind of operator, the one who runs their own open model rather than calling a sealed one over an API (Figure 14). That is a smaller set than the whole market and a particular one: regulated finance, government and defense, sovereign and air-gapped estates, healthcare, and the platform teams that host models for many internal agents. These are the places that already hold the weights for reasons of data control, and for whom a check that runs inside their own process, on their own model, with nothing leaving, is worth more than one that must call out. For an agent on a hosted API this read does not apply, and we say so plainly; the checks on the action and its effect, which do not need the weights, remain the general case there.

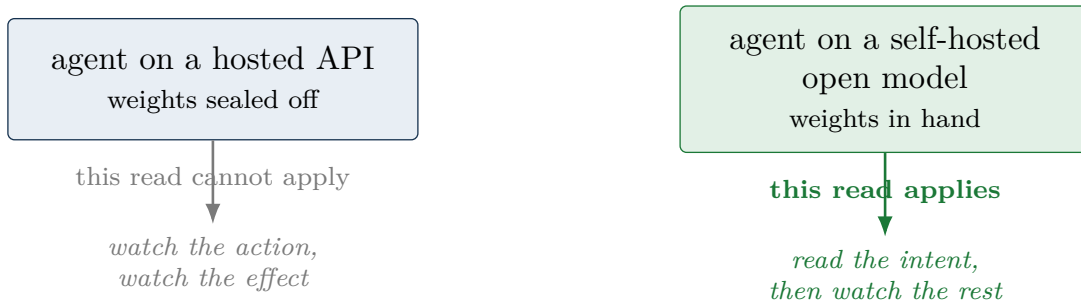


Figure 14. Where the read lives. It needs the weights, so it belongs to the operators who run their own open models: regulated, sovereign, air-gapped, and the platforms that host models for their agents.

11 What it costs and what it misses

We are direct about the edges. The read needs the model’s weights, so it does not reach an agent on a hosted API. The direction belongs to one model and would be recomputed when that model is fine-tuned. The read is one score for the whole request and does not point inside it; we looked for a per-word signal and it was flat, so the read says “this request leans wrong” rather than “here is the wrong part.” Against wording that hides intent in ordinary phrasing it is level with a text classifier, not ahead, so on that cut the win is generality and placement rather than raw accuracy. And the case where reading the inside should matter most, an aligned agent that complies with a hidden instruction while assuring the user it did not, is the one that is hardest to stage in a small model that tends to refuse outright; making it at scale is the next step. None of these unsettle the finding. On a small open model, run on a laptop, an agent’s intent is legible in the activations, before it acts, for the cost of a single dot product.

12 Where this goes

The near work is more models and a maintained probe for each, harder obfuscation to find the point where reading the inside pulls clear of reading the words, and the deception case above, an agent that says one thing and means another, read at the instant it decides. The longer arc is to place this read where an agent platform makes its decisions, so an operator who holds the weights can weigh intent against policy before the action, standing in front of the checks that watch the action and its effect.

Reproducibility. The code, the requests, the probe, and every number in this report are at github.com/kjayashr/the-tell. It runs on one laptop with no GPU, in a few minutes. The

numbers here, 0.993 on Qwen2.5-1.5B and 1.000 on Qwen2.5-3B, come from that pipeline and nowhere else.

The short version is this. We handed an agent's request to a small open model, read a single layer, and knew which way it was leaning before it said a word. We saw that coming.